

SISTEMA DE SEMÁFOROS INTELIGENTES

INFORME TRABAJO PRÁCTICO INTEGRADOR - PyL 2026

Alumnos: Agustin Matias Berton, Nicolas Gustavo Blanco y Milagros Esperanza Gonzalez

Grupo: 3

Fecha de Entrega: 16-06-2026

Objetivos del Trabajo Práctico

- **Modelar un problema del mundo real:** Traducir los requisitos funcionales de un sistema de tráfico urbano a código LISP utilizando el paradigma funcional.
- **Desarrollar autonomía y pensamiento algorítmico:** Resolver la integración de herramientas externas y el aprendizaje de una tecnología nueva con el mínimo de asistencia docente.
- **Análisis Crítico y Metacognición:** Evaluar y tipificar las funciones creadas, comprendiendo sus implicancias en la memoria y el flujo de datos, y comparar cómo diferentes lenguajes abordan un mismo problema lógico.

Fases de desarrollo del proyecto

Fase 1: El problema de los 3 focos

El desarrollo del código fuente fue realizado en common lisp, respetando las restricciones de diseño e implementación propuestas por la cátedra.

El sistema trabaja inicialmente con los tres estados principales del semáforo:

- en-rojo (90 segundos)
- en-verde (120 segundos)
- en-amarillo (6 segundos)

La secuencia original quedó de la siguiente manera:

- **rojo → verde → amarillo → rojo**

Con la secuencia definida, la duración total del ciclo del semáforo fue de 216 segundos.

Diseño Funcional

El sistema fue diseñado evitando el uso de variables globales. El estado del semáforo no se almacena en una variable que cambia, sino que se calcula a partir de los datos recibidos por parámetro.

También se evitaron bucles imperativos como loop, dolist o dotimes. En los casos donde fue necesario recorrer datos, se utilizaron funciones de orden superior como mapcar.

Transiciones del semáforo

La función de transición será válida si un cambio de estado es permitido.

Si el cambio forma parte de la secuencia definida, devuelve una lista con el estado actual y la acción correspondiente. Si el cambio no está permitido, devuelve una acción por defecto.

Esto permite evitar transiciones incorrectas, como pasar directamente de rojo a amarillo sin respetar el orden lógico del semáforo.

Temporizador Automático

Para automatizar el funcionamiento del semáforo se implementó la función timer.

Esta función recibe como parámetro un tiempo Unix y determina qué color del semáforo corresponde en ese momento. Para lograrlo, se utiliza la función mod, que permite obtener el resto de dividir el tiempo recibido por la duración total del ciclo.

En esta primera versión, el ciclo completo tiene una duración de 216 segundos, distribuidos de la siguiente manera:

- en-rojo: 0 - 89 segundos
- en-verde: 90 - 209 segundos
- en-amarillo: 210 - 215 segundos

Auditoría del sistema

Se implementó una función de auditoría para detectar cambios de luz.

La función compara el color correspondiente al segundo anterior con el color del segundo actual usando timer.

Si los colores son iguales, informa que no hubo cambio. Si son distintos, muestra el cambio realizado.

Análisis de Ciclos

Para analizar el ciclo del semáforo se calculó la duración total sumando los tiempos de cada color: $90 + 120 + 6 = 216$ segundos.

También se agregó una función que evalúa si el ciclo está dentro del rango recomendado.

Como el ciclo dura 216 segundos, el sistema lo clasifica como demasiado largo, ya que supera los 150 segundos indicados como límite superior.

Distribución Temporal

La distribución temporal calcula qué porcentaje del ciclo ocupa cada color.

En esta primera versión, el ciclo total dura 216 segundos:

- rojo: 90 segundos
- verde: 120 segundos
- amarillo: 6 segundos

Los porcentajes aproximados son:

- rojo: 41,66%
- verde: 55,55%
- amarillo: 2,77%

Iteración 2

En la Iteración 2 se extendió la versión inicial del sistema agregando una mejora en la secuencia del semáforo.

El principal cambio fue incorporar el estado en-intermitente, que representa una pausa de seguridad entre los cambios de luz.

La secuencia del semáforo pasó a ser:

- **rojo** → **intermitente** → **verde** → **intermitente** → **amarillo** → **intermitente** → **rojo**

Con este cambio, la duración total del ciclo pasó de 216 a 225 segundos, ya que se agregaron tres periodos intermitentes de 3 segundos cada uno.

También se actualizaron las funciones transición y timer para obtener el nuevo estado. De esta forma, las transiciones ya no pasan directamente de un color a otro, sino que atraviesan primero el estado intermitente.

Además, se actualizó el cálculo de duración del ciclo, la distribución temporal y la cantidad de ciclos por tiempo, para que todos los resultados coincidieran con la nueva duración total.

Por último, se agregó la función informe, que permite guardar registros de ejecución en un archivo de texto. Para mejorar la lectura de las fechas se utilizó la librería local-time, cargada mediante Quicklisp.

Fase 2: Autonomía y Ecosistema

En la segunda fase se investigó el uso de Quicklisp, el gestor de paquetes de Common Lisp.

Quicklisp permitió cargar una librería externa dentro del proyecto. La librería elegida fue local-time, ya que permite trabajar con fechas y horarios de manera más legible.

Uso de local-time

La librería local-time fue utilizada para mejorar el sistema de auditoría y generación de informes.

En lugar de registrar únicamente un número de tiempo Unix, el sistema puede convertir ese valor en una fecha comprensible para una persona.

Persistencia de datos

Como extensión, se implementó una función informe, encargada de guardar los registros de transición en un archivo de texto llamado:

informe-ejecucion-semaforo.txt

La función utiliza with-open-file para abrir o crear el archivo y format para escribir los datos.

Además, utiliza mapcar para recorrer los registros sin usar bucles imperativos.

Fase 3: Estudio Comparativo

En la tercera fase se trabajó con Scheme, el lenguaje asignado al grupo para realizar la comparación con las funciones solicitadas (transición y timer).

Scheme al ser de la familia Lisp, comparte varias características de Common Lisp, sin embargo también contiene sus diferencias.

Diferencias entre Common Lisp y Scheme

Al ser Scheme un Lisp-1 y Common Lisp un Lisp-2, esta última maneja las funciones y variables en espacios de nombres separados, en cambio Scheme comparte un mismo espacio de nombres. Por esta razón en Scheme no es necesario usar `funcall` ni `#'` para trabajar con funciones como valores. Las funciones pueden pasarse directamente como argumentos, de una forma más simple.

A su vez también tiene diferencia de sintaxis como `defun/define`, `eq/eq?` o `mod/modulo`.

En esta fase también se analizó la recursividad de cola. Scheme exige la optimización de llamadas de cola, lo que permite que una función recursiva pueda ejecutarse sin consumir memoria de pila innecesariamente cuando la llamada recursiva es la última operación.

Bitácora de Depuración y Errores

Error 1: Uso de parámetro como función

En una primera versión de “recomendación-ciclo”, se escribió algo parecido a:
(< (duración) 35)

Causa: duración era un parámetro y no una función. Al escribir (duración), el programa intenta ejecutarlo como función

Solución: Se corrigió usando el parámetro directamente (< duracion 35)

Error 2: Inconsistencia en el nombre del estado intermitente

Causa: Durante la implementación de la Iteración 2, algunas funciones no usaban el mismo nombre para representar el estado intermitente. Por ejemplo, una función podía devolver un símbolo distinto al que otra función esperaba recibir como entrada. Esto hacía que algunas transiciones válidas no fueran reconocidas correctamente y terminaran en la acción por defecto.

Solución: Se unificó el nombre del estado intermitente en todo el sistema, utilizando el símbolo ``en-intermitente`` en las funciones afectadas. De esta manera, ``timer``, ``transicion``, auditoría y distribución temporal trabajan con el mismo criterio.

Error 3: Acumulación de registros en el archivo de informe

Causa: En la función ``informe``, inicialmente se usó la opción ``:append`` dentro de ``with-open-file``. Esto hacía que cada vez que se ejecutaba la función, el nuevo informe se agregara al final del archivo anterior.

El problema era que, al hacer varias pruebas, el archivo quedaba con registros repetidos o acumulados, dificultando la lectura del informe final.

Solución: Se cambió la opción ``:append`` por ``:supersede``, para que en cada ejecución se genere un informe nuevo, reemplazando el contenido anterior del archivo.

Error 4: Duración del ciclo desactualizada

Al principio el ciclo del semáforo duraba 216 segundos: $90 + 6 + 120 = 216$

Después agregamos el estado en-intermitente, pero algunas funciones seguían usando 216 segundos.

Causa: El timer ya había sido actualizado a 225 segundos, pero otras funciones como `distribucion-temporal` o los comentarios todavía tenían el valor anterior.

Solución: Actualizamos el ciclo completo a 225 segundos: $90 + 3 + 6 + 3 + 120 + 3 = 225$

También se corrigieron los cálculos de distribución temporal y duración del ciclo.

Organización del Repositorio

La gestión del código se centralizó en un repositorio de GitHub con la siguiente estructura:

TPI-Funcional-2026-Grupo3/

```
|— lisp/
|   |— core.lisp
|— comparativa/
|   |— solucion.scm
|— docs/
|   |— INFORME.pdf
|   |— HONOR.md
|— README.md
```

Se emplearon commits periódicos para documentar la evolución del sistema y demostrar el avance de los aportes realizados por cada integrante.

Conclusión

La realización de este proyecto facilitó la aplicación práctica del paradigma funcional sobre una problemática cotidiana, logrando modelar la gestión de tráfico mediante el empleo de símbolos, listas y funciones en Common Lisp.

A lo largo de las distintas etapas, se evolucionó desde un prototipo inicial hacia una versión extendida que incorporó estados de intermitencia, mecanismos de persistencia y la integración de la librería externa `local-time` mediante `Quicklisp`.

Asimismo, el análisis comparativo desarrollado con Scheme resultó fundamental para identificar las discrepancias operativas y sintácticas existentes entre los diversos dialectos de la familia Lisp.

Como cierre, el trabajo permitió consolidar el entendimiento sobre la resolución de problemas lógicos bajo restricciones funcionales, garantizando una implementación modular, documentada y eficiente.